From Blind Search to Memory-Aware Evolution: Efficient DBMS Tuning via Collaborative Diagnosis and Utility-Aware Retrieval (Appendix)

Anonymous Author(s)

OUTLINE

This supplemental material is organized as follows.
A. Prompt Templates for EvoTune Agents.
B. Effect of Tuning Subspace Formulation.
C. Comparison of Different LLM APIs.
D. Detailed Overhead Breakdown.

A Prompt Templates for EvoTune Agents

This section presents the prompt templates used in EvoTune.

A.1 Detection Agent Prompt

Listing 1 shows the prompt template used by the Detection Agent. It takes as input the collected query information and trajectory-level tuning feedback, and outputs the target tuning components.

Listing 1: Prompt template for tuning space localization.

Template: Detection Agent

You are a database performance diagnosis expert. Your task is to identify the most promising tuning component space(s) for the following slow SQL query. To do so, you should reason directly from the current query context, trajectory-level tuning feedback, and DBA-style knowledge, and determine which component space(s) are most likely to deliver substantial performance improvement.

Candidate Tuning Components
The allowed components are exactly: [index, knob, hint, query]

Context
1. Query Information
- SQL Statement: <SQL_QUERY>
- Execution Plan: <EXECUTION_PLAN_JSON>
- Execution Runtime: <EXECUTION_RUNTIME>
- Internal Metrics:
  - Tuples Returned: <TUPLES_RETURNED>
  - Blocks Hit: <BLOCKS_HIT>
  - Blocks Read: <BLOCKS_READ>
  - Tuples Fetched: <TUPLES_FETCHED>
  - Index Tuples Fetched: <INDEX_TUPLES_FETCHED>
  - Sequential Tuples Read: <SEQUENTIAL_TUPLES_READ>
  - Sequential Scans: <SEQUENTIAL_SCANS>
  - Index Scans: <INDEX_SCANS>
  - Heap Blocks Hit: <HEAP_BLOCKS_HIT>
  - Heap Blocks Read: <HEAP_BLOCKS_READ>
  - Index Blocks Hit: <INDEX_BLOCKS_HIT>
  - Index Blocks Read: <INDEX_BLOCKS_READ>

2. Database Environment Information
- Database Type: <DB_TYPE>
- Workload Type: <WORKLOAD_TYPE>
- Database Size: <DATABASE_SIZE>
- Database Schema: <DATABASE_SCHEMA>
- CPU: <CPU_USAGE>
- Read I/O: <READ_IO>
- Write I/O: <WRITE_IO>
- Virtual Memory: <VIRTUAL_MEMORY>
- Physical Memory: <PHYSICAL_MEMORY>
- Net Received: <NET_RECEIVED>
- Net Sent: <NET_SENT>

Reasoning Guidance
1. Historical Tuning Summary
<TUNING_TRAJECTORY>

2. Trajectory-Guided Reasoning
Guide component reasoning according to the following principles:
- If `success_count>0` and `fail_count=0`: prioritize component since past interventions consistently validate it as a bottleneck.
- If `success_count=0` and `fail_count>0`: deprioritize the component and encourage exploration of alternative components, since previous interventions have not led to improvement.
- If `success_count>0` and `fail_count>0`: keep the component under consideration and make a more nuanced judgment based on DBA knowledge and the available evidence.

3. DBA Knowledge-Guided Reasoning
Use DBA-style knowledge to reason over the query context to identify target tuning components.
- **Plan suboptimality**: A plan bottleneck typically arises from suboptimal access paths, join orders, or join strategies that introduce unnecessary overhead, such as redundant sorting, full table scans, or excessive intermediate results. Typical evidence includes cases where the optimizer prefers sequential scans despite the presence of indexes under high selectivity, incurs extra sorting due to unordered access paths, or exhibits hash join spill-to-disk behavior...
- **Index bottlenecks**: Index bottlenecks are characterized by large-range scans on selective predicates. Typical evidence includes a large gap between scanned and returned rows, predicates that cannot be effectively exploited by indexes, and repeated heap/table access due to non-covering indexes...
- **Knob misconfiguration**: Knob tuning becomes important when analytical operators spill to disk or lack sufficient memory or parallel resources. Typical evidence includes disk-backed temporary tables, underutilized parallel workers, and high-cost join or aggregation execution under limited resources...
- **Query formulation issues**: Query rewriting is prioritized when the SQL structure itself introduces unnecessary work. Typical evidence includes implicit type conversions (blocking B-Tree access and triggering full scans) and overlong `IN` lists...

Constraints (STRICT)
- You MUST select tuning components only from the following labels: `index`, `knob`, `hint`, `query`.

- You MUST output strict JSON only, containing a single field `predicted_components`. No extra text and no explanation.
- `predicted_components` MUST be an array of the final selected tuning components.

Output JSON Format:
`{"predicted_components": ["index","knob"]}`

A.2 Prompt-Guided Subspace Formulation

After localizing the target tuning components, EvoTune performs prompt-guided construction and refinement of the corresponding component-wise spaces using historical observations. The resulting prompt snippets, derived only from these selected spaces and their refinement signals, are then assembled into the `<ACTION_SPACE>` field of the Tuning Agent Prompt.

Base Space Formulation. EvoTune first initializes the tuning space according to the component(s) $\mathcal{B} \subseteq \{\text{index}, \text{query}, \text{knob}, \text{hint}\}$. For the index space Θ_{index}, EvoTune derives candidate indexes from the database schema for the current query, including composite indexes when supported by the query structure and execution plan, while taking existing indexes in the DBMS into account to avoid redundant or subsumed creations. This space is then provided to the Tuning Agent as Listing 2.

Listing 2: Prompt snippet for basic index-space.

Template: Index-Space Specification

- Index Space:
- Use `CREATE INDEX IF NOT EXISTS` or `DROP INDEX IF EXISTS`, and name each newly created index using the pattern `<table_name>_<col>_idx`.
- Derive query-specific index candidates from the database schema, and consider composite indexes when supported by the query structure and execution plan, rather than making generic index recommendations.
- Given the existing indexes in the DBMS (`<EXISTING_INDEXES>`), avoid creating duplicate indexes, and do not drop primary keys, unique indexes, or any indexes that enforce constraints.

For query rewriting space Θ_{query}, EvoTune builds upon the rewrite rules provided in [1]. Specifically, it identifies applicable rewrite rules through a predefined matching procedure that checks whether the structural patterns of the current SQL satisfy the rule conditions. The resulting rewrite space is injected into the prompt as follows in Listing 3.

Listing 3: Prompt snippet for basic query-rewrite space.

Template: Query-Rewrite Space Specification

- Query Rewrite Space:
- Query rewriting must generate a semantically equivalent `SELECT` statement that preserves the original result set.
- Consider only the rewrite rules provided below as the initial rewrite space: `<APPLICABLE_REWRITE_RULES>` . Apply only rules that are applicable to the current query, and prioritize rewrites that simplify execution, eliminate redundant operations, or improve data access efficiency.

For knob tuning space Θ_{knob} and optimizer hint space Θ_{hint}, EvoTune organizes the tunable knobs and hints into structured lists, where each item includes its allowable value range or options, default setting, and current runtime configuration in the DBMS, as shown in Listing 4 and Listing 5.

Listing 4: Prompt snippet for basic knob-space.

Template: Knob-Space Specification

- Knob Space:
- Adjust database configuration parameters using `SET knob = value` and focus on parameters that can improve query performance.
- Consider the following knob list as the initial tuning space: `<KNOB_LIST>`
- For each relevant knob, consider its allowed value range, default setting, and current configuration when making recommendations.

Listing 5: Prompt snippet for basic hint-space.

Template: Hint-Space Specification

- Hint Space:
- Output ONLY the hint block like: `/*+ Set(knob_name value) ... */`. Do NOT include the SQL query when using hints.
- Consider the following hint list as the initial tuning space: `<HINT_LIST>`
- For each relevant hint, consider its default setting and current configuration when making recommendations.

Space Pruning and Refinement. EvoTune refines each basic tuning space using the retrieved observations $\mathbb{O}$ and their execution feedback $\mathcal{I}$ to derive an evidence-informed recommendation space. The resulting space serves as guidance for the Tuning Agent rather than a hard constraint. Component-specific refinement strategies are applied as follows:

1. Discrete Component Pruning (Index & Query). For discrete components, we aim to exclude candidates that have historically failed to meet a minimum benefit threshold ρ. Let $\bar{\mathcal{I}}(x)$ denote the average percentage improvement of a candidate x recorded in the observation set $\mathbb{O}$. We identify the sets of ineffective candidates for indexes and query rules, denoted as $\mathcal{R}_{\text{index}}$ and $\mathcal{R}_{\text{query}}$, respectively:

$$\mathcal{R}_{\text{index}} = \left\{x \in \mathbb{O}_{\text{index}} \mid \bar{\mathcal{I}}(x) \le \rho\right\}, \mathcal{R}_{\text{query}} = \left\{x \in \mathbb{O}_{\text{query}} \mid \bar{\mathcal{I}}(x) \le \rho\right\}. \tag{1}$$

During the tuning phase, EvoTune provides the Tuning Agent with pruning guidance based on the refined spaces in Listing 6 and Listing 7. For discrete components, the refined spaces are obtained by removing historically ineffective candidates from the base spaces:

$$\Theta^*_{\text{index}} = \Theta_{\text{index}} \setminus \mathcal{R}_{\text{index}}, \quad \Theta^*_{\text{query}} = \Theta_{\text{query}} \setminus \mathcal{R}_{\text{query}}. \tag{2}$$

For index recommendation, EvoTune injects historically ineffective indexes as shown in Listing 6.

Listing 6: Prompt snippet for index-space pruning.

Template: Index-Space Pruning Guidance

- Index Pruning: Based on historical failure cases with the same tuning component, the following indexes have been proven ineffective, please avoid using them: `<FAILED_INDEX_NAMES>`

For query rewriting, EvoTune injects historically ineffective rewrite rules as shown in Listing 7.

Listing 7: Prompt snippet for query-rewrite pruning.

Template: Query-Rewrite Pruning Guidance

- Query Pruning: Based on historical failure cases with the same tuning component, the following rewrite rules have been proven ineffective, please avoid using them: `<FAILED_REWRITE_RULES>`

2. Hint Space Refinement. EvoTune focuses on *enable_knobs* for optimizer hints (binary variables). For each hint h, we compute the average improvement $\bar{\mathcal{I}}_{h,v}$ for value $v \in \{0, 1\}$. To prune the search space, we define the refined domain D_h^* for each hint h:

$$D_h^* = \begin{cases} \{\arg\max_v \bar{\mathcal{I}}_{h,v}\}, & \text{if } |\bar{\mathcal{I}}_{h,1} - \bar{\mathcal{I}}_{h,0}| > \rho, \\ \{0, 1\}, & \text{otherwise.} \end{cases} \tag{3}$$

The refined hint space is constructed as $\Theta^*_{\text{hint}} = \prod_{h=1}^{H} D_h^*$.

Listing 8: Prompt snippet for hint-space refinement.

Template: Hint-Space Refinement Guidance

- Hint Refinement: Based on historical success cases with the same tuning component, the following hint settings have been proven effective, suggest prioritizing exploration: `<POSITIVE_HINTS>`

3. Knob Space Refinement. EvoTune contracts the knob space Θ_{knob} to a *Promising Region* based on high-quality historical data. Let $\mathbf{x} \in \Theta_{\text{knob}}$ be a configuration vector, and x_k be the value of the k-th knob dimension. First, we filter the historical observations $\mathbb{O}_{\text{knob}}$ to identify effective configurations $\mathcal{K} = \left\{\mathbf{x} \in \mathbb{O}_{\text{knob}} \mid \bar{\mathcal{I}}(\mathbf{x}) > \rho\right\}$. To prevent over-pruning due to data sparsity, we apply a count threshold $n_{\min}$. For each knob dimension k, the refined domain D_k^* is derived as:

$$D_k^* = \begin{cases} [\min_{\mathbf{x}\in\mathcal{K}} x_k, \max_{\mathbf{x}\in\mathcal{K}} x_k], & \text{if } k \text{ is continuous} \wedge |\mathcal{K}| \geq n_{\min}, \\ \{x_k \mid \mathbf{x} \in \mathcal{K}\}, & \text{if } k \text{ is categorical} \wedge |\mathcal{K}| \geq n_{\min}, \\ D_k^{\text{base}}, & \text{otherwise,} \end{cases} \tag{4}$$

where D_k^{base} represents the original domain of the k-th knob. The recommended knob region can be represented as $\Theta^*_{\text{knob}} = \prod_{k=1}^{K} D_k^*$ over all K dimensions.

Listing 9: Prompt snippet for knob-space refinement.

Template: Knob-Space Refinement Guidance

- Knob Refinement: Based on historical success cases with the same tuning component, the following knobs and recommended value ranges have been proven effective; prioritize exploration accordingly: `<KNOB_NAME, RANGE, NUM_CASES>`

Finally, EvoTune constructs the refined global tuning space

$$\Theta^* = \prod_{b\in\mathcal{B}} \Theta_b^*, \tag{5}$$

where only the refined subspaces corresponding to the localized component set $\mathcal{B}$ are retained. These selected subspaces are then instantiated and assembled into the `<ACTION_SPACE>` field of the Tuning Agent Prompt, providing a compact and evidence-informed recommendation space that guides action generation toward higher-utility regions without strictly restricting it.

A.3 Tuning Agent Prompt

Listing 10 presents the prompt template used by the Tuning Agent. The template incorporates the current query context, localized bottleneck component(s), retrieved positive and negative historical cases, and the refined action space.

Listing 10: Prompt template for action recommendation.

Template: Tuning Agent

You are an SQL optimization expert. The bottleneck-related tuning components have already been localized. Your job is to recommend executable tuning action(s) **only within the localized tuning subspace**, using the current query context, retrieved historical observations with experiences, and the refined action space.

Context

1. Query Information

- SQL: `<SQL_QUERY>`
- Execution Plan: `<EXECUTION_PLAN_JSON>`
- Execution Runtime: `<EXECUTION_RUNTIME>`
- Internal Metrics:
  - Tuples Returned: `<TUPLES_RETURNED>`
  - Blocks Hit: `<BLOCKS_HIT>`
  - Blocks Read: `<BLOCKS_READ>`
  - Tuples Fetched: `<TUPLES_FETCHED>`
  - Index Tuples Fetched: `<INDEX_TUPLES_FETCHED>`
  - Sequential Tuples Read: `<SEQUENTIAL_TUPLES_READ>`
  - Sequential Scans: `<SEQUENTIAL_SCANS>`
  - Index Scans: `<INDEX_SCANS>`
  - Heap Blocks Hit: `<HEAP_BLOCKS_HIT>`
  - Heap Blocks Read: `<HEAP_BLOCKS_READ>`
  - Index Blocks Hit: `<INDEX_BLOCKS_HIT>`
  - Index Blocks Read: `<INDEX_BLOCKS_READ>`

2. Database Environment Information

- Database Type: `<DB_TYPE>`
- Workload Type: `<WORKLOAD_TYPE>`
- Database Size: `<DATABASE_SIZE>`
- Database Schema: `<DATABASE_SCHEMA>`
- CPU: `<CPU_USAGE>`
- Read I/O: `<READ_IO>`
- Write I/O: `<WRITE_IO>`
- Virtual Memory: `<VIRTUAL_MEMORY>`

- Physical Memory: <PHYSICAL_MEMORY>
- Net Received: <NET_RECEIVED>
- Net Sent: <NET_SENT>

3. Localized Bottleneck Component(s)
<TARGET_COMPONENTS>

4. Tuning Mode
<MODE_INSTRUCTION>

5. Retrieved Historical Observations and Experiences
- Positive Observations (historically effective): <POSITIVE_CASES>
- Negative Observations (historically ineffective): <NEGATIVE_CASES>
- Experiences: <EXPERIENCE_SUMMARY>

6. Reflective Reasoning Guidance
Before proposing your optimization solution, analyze the historical observations and experiences to understand why certain approaches succeeded or failed. Adapt successful patterns to the current context while avoiding patterns that failed in similar situations.

7. Refined Tuning Subspace
<ACTION_SPACE>

Constraints
- The tuning action must be directly executable in <DB_TYPE>.
- The tuning action must only operate on the component(s) listed in <TARGET_COMPONENTS>.
- The tuning action should be safe and must not drop or delete data.

Output Format
You must output *only* the following format:
```
Tuning_Action:<executable tuning action(s) or an empty string>
Query_Rewrite:<yes or no>
```

Output Requirements
- Set `Query_Rewrite` to `yes` if the selected action(s) involve query rewriting, and place the full rewritten SQL in `Tuning_Action`; otherwise, set `Query_Rewrite` to `no`.
- If multiple executable statements are needed, place all of them in `Tuning_Action` in executable form.

B Effect of Tuning Subspace Formulation.

We investigate the impact of space formulation strategies by comparing three configurations: (1) *No Formulation*, (2) *No Pruning*, and (3) *Formulation*. As illustrated in Figure 1, *No Formulation* suffers from high dimensionality, resulting in the slowest convergence and stabilizing at a high execution time. While introducing a structured action space (*No Pruning*) improves performance, the most significant gain is achieved by the complete *Formulation* strategy. By leveraging historical pruning to filter low-utility candidates, the full formulation accelerates convergence rapidly within the first 6 hours of tuning and achieves the lowest execution time.

C Comparison of Different LLM APIs

Figure 2 compares the performance of different LLM backends. We observe that stronger models do not always yield better tuning results. GPT-4.1 performs best, achieving a favorable balance between reasoning quality and response latency. By contrast, GPT-5 performs worst: as shown in Table 1, its high latency and verbosity introduce substantial over-reasoning overhead, reducing the number of valid tuning iterations within the time budget. GPT-3.5 is the most cost-effective option, but its limited reasoning depth leads to an earlier performance bottleneck.

Table 1: LLM APIs Performance Summary

Metric	GPT-3.5-turbo	GPT-4.1	GPT-5
Avg Request Time (s)	2.13	2.65	44.57
Avg Completion Tokens (t)	168.97	136.79	3786.20
Avg Cost ($)	0.0029	0.0144	0.0454
Performance (s)	67.49	52.16	79.81

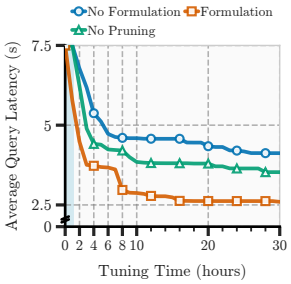


Figure 1: Space Analysis.

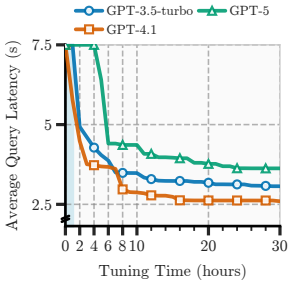


Figure 2: LLM APIs Analysis.

Table 2: Algorithm (A), replay (R), and total (T) overheads in hours to reach different performance targets.

Workload	Method	$0.3I^*$			$0.5I^*$			$0.8I^*$		
		A/h	R/h	T/h	A/h	R/h	T/h	A/h	R/h	T/h
JOB	EvoTune	0.12	0.17	**0.29**	0.12	0.17	**0.29**	0.99	4.75	**5.74**
	Booster	0.22	0.60	0.82	0.73	4.64	5.37	–	–	–
	Proto-X	0.07	0.83	0.90	0.47	5.36	5.83	1.30	15.00	16.30
	UniTune	0.02	1.54	1.56	0.57	7.05	7.62	–	–	–
TPC-H	EvoTune	0.12	0.46	**0.58**	0.12	0.46	**0.58**	0.12	0.46	**0.58**
	Booster	0.06	0.68	0.74	0.46	0.87	1.33	0.46	0.87	1.33
	Proto-X	0.05	1.14	1.19	0.05	1.14	1.19	0.05	1.14	1.19
	UniTune	0.18	2.83	3.01	0.18	2.83	3.01	0.18	2.83	3.01
TPC-DS	EvoTune	1.42	5.37	6.79	1.42	5.37	**6.79**	1.42	5.37	**6.79**
	Booster	2.74	4.42	7.16	2.74	4.42	7.16	10.44	11.59	22.03
	Proto-X	0.69	7.93	8.62	0.69	7.93	8.62	–	–	–
	UniTune	0.42	5.52	**5.94**	0.51	26.20	26.71	–	–	–

D Detailed Overhead Breakdown

Table 2 reports the algorithm and replay overhead required by each method to reach different performance targets. Here, L_0 and L denote the average query latency under the initial configuration and the best configuration found so far, respectively; $I = (L_0 - L)/L_0$ denotes the relative latency reduction, and I^* is the best reduction achieved within the tuning budget. Overall, EvoTune reaches the same performance targets with substantially lower total overhead than other tuning methods, indicating that its space localization and action recommendation effectively reduce costly replay. Although EvoTune introduces additional algorithm overhead, this cost is often offset by the reduction in replay overhead, enabling it to reach strong performance targets earlier.

References

[1] Zhaoyan Sun. 2025. R-Bot: An LLM-based Query Rewrite System. https://github.com/curtis-sun/LLM4Rewrite/blob/main/explain_rule/calcite_rewrite_rules_structured.jsonl Accessed: Dec. 26, 2025.